

**DEV2 – JAVL – Laboratoire Java****TD 13 – MVC - Refactoring***MVC - Refactoring***Résumé**

Dans ce TD, nous allons réviser notre version mini-mini du jeu de société *Scrabble*.

Ce TD a pour objectif de présenter le patron d'architecture (*architectural pattern*) MVC; modèle, vue, contrôleur (*model, view, controller*) et de montrer comment notre IDE peut nous aider à refactoriser du code.

**Table des matières**

|          |                                                                                            |           |
|----------|--------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Présentation</b>                                                                        | <b>2</b>  |
| <b>2</b> | <b><i>Refactoring</i></b>                                                                  | <b>3</b>  |
| 2.1      | La classe <code>Letter</code> . . . . .                                                    | 3         |
| 2.2      | Les classes <code>Bag</code> , <code>Direction</code> et <code>Dictionary</code> . . . . . | 4         |
| <b>3</b> | <b>Le modèle, la vue et le contrôleur</b>                                                  | <b>6</b>  |
| 3.1      | La classe <code>Model</code> , point d'entrée du modèle . . . . .                          | 6         |
| 3.2      | La classe <code>View</code> , une première vue du modèle . . . . .                         | 8         |
| 3.3      | La classe <code>Controller</code> , le contrôleur . . . . .                                | 9         |
| <b>4</b> | <b>Une autre vue</b>                                                                       | <b>11</b> |

## 1 Présentation

Modèle-vue-contrôleur (*model-view-controller*) est une architecture divisée en 3 parties :

- ▷ le **modèle** qui contient la logique métier. Le modèle traite les données de l'application et s'assure de la cohérence métier ;
- ▷ les ou la **vue** qui se charge de la représentation visuelle de l'état du modèle ;
- ▷ les ou le **contrôleur** qui reçoit les interactions de l'utilisateur ou de l'utilisatrice et les transmet au modèle.

Ce patron d'architecture suit le principe de séparation des responsabilités (*SOC, separation of concerns*) ; chaque partie a sa responsabilité et une modification apportée à l'une d'entre elle ne devrait — idéalement — pas avoir d'impact sur les autres.

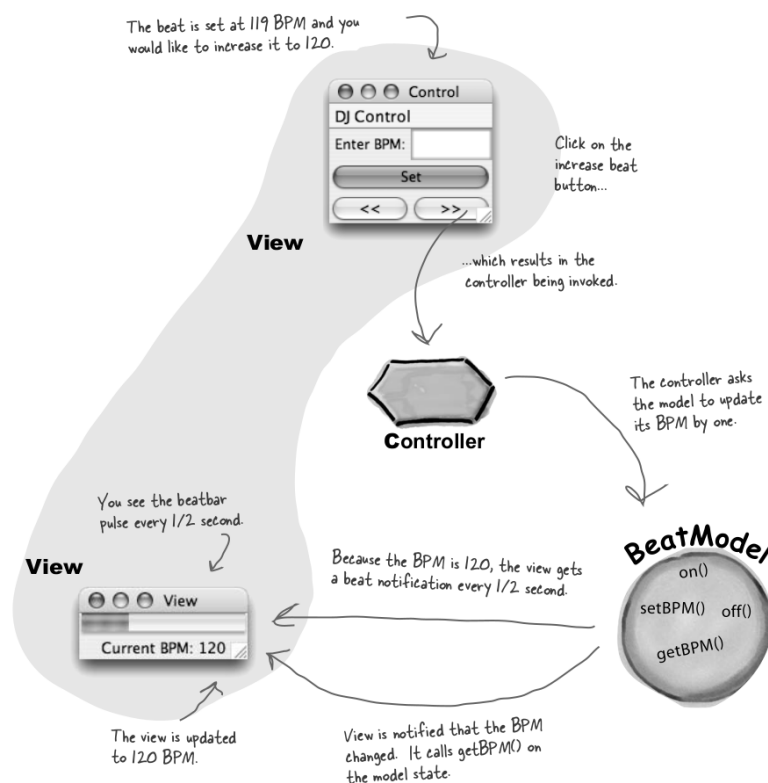


FIGURE 1 – Source O'Reilly, Head First Design Pattern p 536

La figure 1 illustre cette architecture dans le cadre d'une application représentant un lecteur MP3 pour un DJ. Lorsqu'une musique est jouée, le player joue le son, montre l'état d'avancement de la lecture, permet de changer le BPM (*beats per minute*)...

Via l'interface fournie par une **vue**, l'utilisateur ou l'utilisatrice demande par exemple d'augmenter ou diminuer le *beat*. Le **contrôleur** traduit cette demande pour qu'elle soit compréhensible par le **modèle**. Le modèle s'exécute, joue le morceau et signale à la **vue** chaque fois que son état change : la durée restante, le PBM actuel... La vue se met à jour et affiche cet état pour l'utilisateur ou l'utilisatrice.

De manière plus générale sur la partie gauche de la figure 2 page suivante ; l'utilisatrice ou l'utilisateur demande au contrôleur de *jouer un morceau de musique*. Cette demande se fait soit directement, soit par le biais d'un bouton dans une fenêtre graphique. Le contrôleur interprète cette demande pour qu'elle soit compréhensible par le modèle. Le

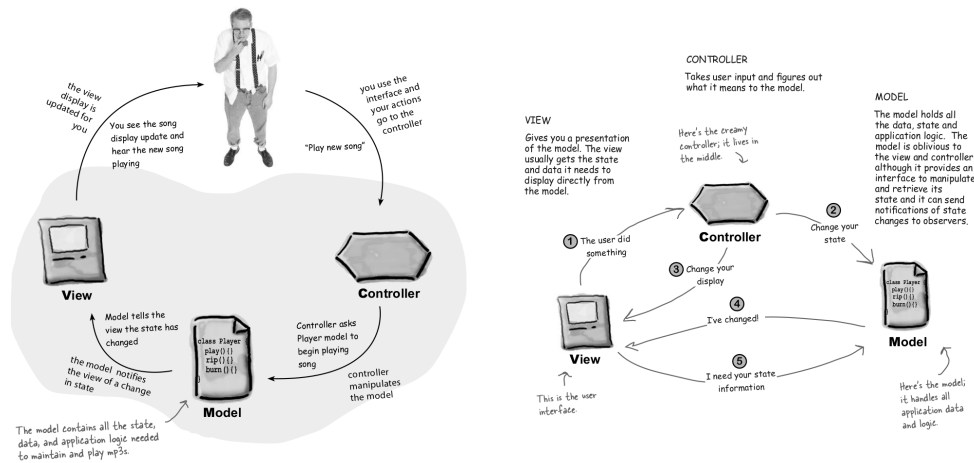


FIGURE 2 – Source O'Reilly, Head First Design Pattern p 529 et 530

modèle s'exécute et notifie lorsque son état change. Les ou la vue est réceptive à ces notifications et se met à jour.

La partie droite de la figure 2 s'explique comme suit :

1. L'utilisateur ou l'utilisatrice interagit avec la vue. Une interaction de l'utilisateur, comme un clic de souris, est transmise au contrôleur qui va la traduire en une demande (ou plusieurs) au modèle.
2. Le contrôleur demande au modèle d'exécuter la demande et, dans la plupart des cas, de modifier son état.
3. Le contrôleur peut également demander à la vue de changer ce qu'elle affiche pour signaler que la demande est prise en charge, pour qu'un bouton ne soit plus disponible pendant l'exécution de la demande...
4. Lorsque le modèle a terminé, il informe — on dit *notifier* — la ou les vues que son état a changé... et qu'il faudra sans doute se mettre à jour.
5. Suivant l'implémentation du patron d'architecture, la vue peut recevoir l'état du modèle ou lui demander afin de se mettre à jour.

Source O'Reilly Head First Design Pattern

## 2 Refactoring

Le *refactoring* (ou réusinage en français puritain) est l'action qui consiste à modifier un code pour qu'il soit plus efficace, pour adapter sa structure, pour le rendre plus lisible, pour corriger un *bug*...

Nous allons refactoriser notre code du **Scrabble** pour lui donner une structure MVC *mais* avant ça, vérifions l'état initial du projet classe par classe.

### 2.1 La classe Letter

La classe **Letter** a deux attributs — la lettre qu'elle représente et la valeur associée — et deux *getters*\*.

Elle n'a pas de *setters*. En effet, par exemple, la lettre  $B_3$  reste cette lettre durant tout le jeu.

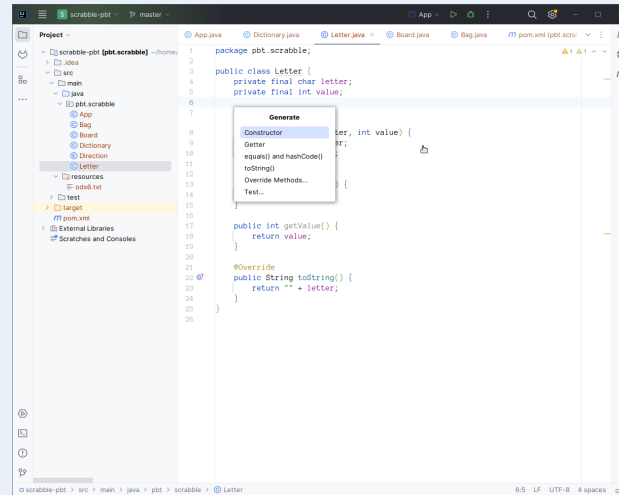
\*. Accesseurs en français. Dans la suite, nous utiliserons toujours le terme anglais

Les attributs d'une lettre ne peuvent être modifiés; il n'y a pas de *setters* et ils sont privés. Ils peuvent donc recevoir le mot clé `final`<sup>†</sup>.

### Tip

La combinaison de touche **[Alt][Insert]** est un raccourci pour ouvrir le menu contextuel *Generate*. Ce menu propose de générer rapidement : constructeur, *getters*, *setters*, surcharge de méthodes...

Ce menu est également accessible — comme tout menu contextuel — par un clic droit.



## 2.2 Les classes Bag, Direction et Dictionary

La classe *Direction* n'a besoin d'aucun changement.

Si vos lettres avaient déjà deux attributs, la classe *Bag* ne nécessite aucun changement. Dans le cas contraire, il faudra attribuer la bonne valeur à chaque lettre.

La classe *Dictionary* ne nécessite aucun changement non plus. Si vous ne l'aviez pas encore ajoutée, ajoutez-là.

Vous trouverez à la figure 3 page suivante un diagramme de classes représentant le projet à ce stade.

<sup>†</sup>. Il est plus que probable que l'IDE propose l'ajout de `final` car il aura détecté que les valeurs ne changent pas.

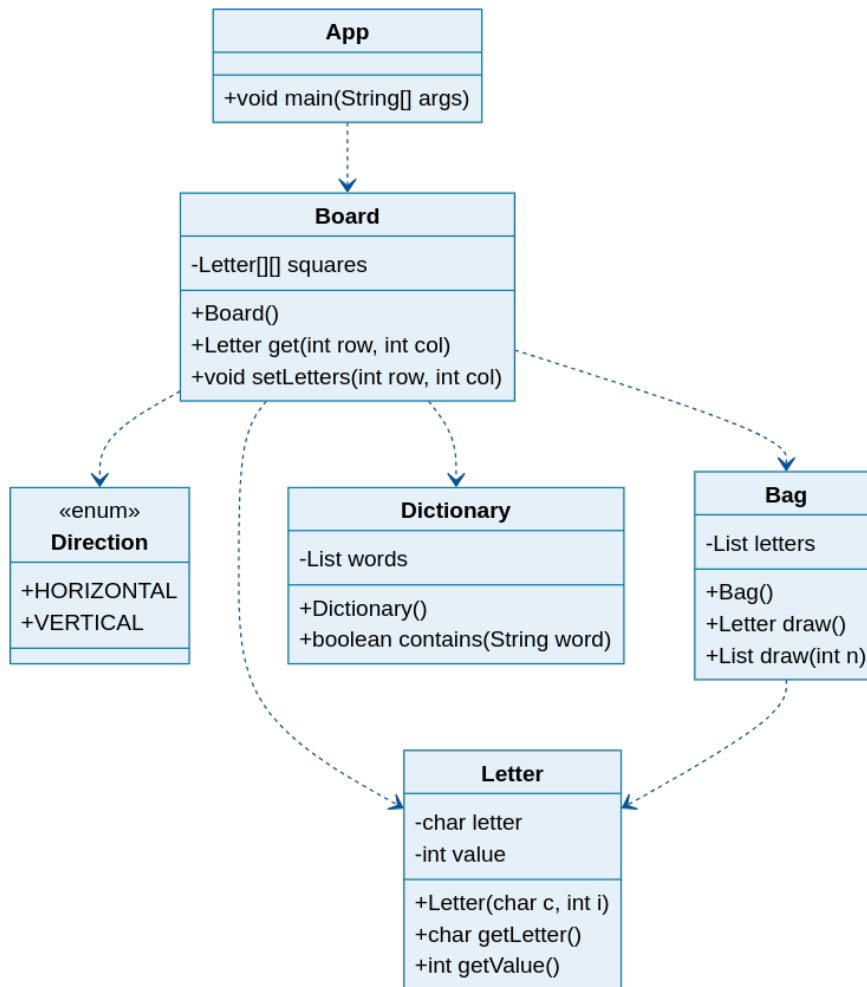
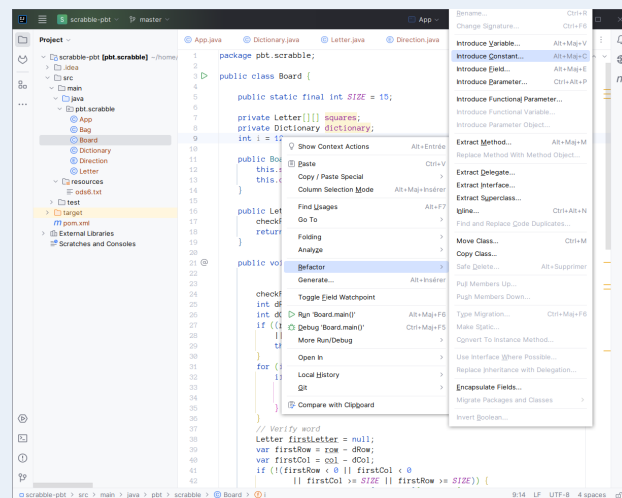


FIGURE 3 – Diagramme de classes (avant refactorisation du projet)

### Tip

Un autre menu contextuel intéressant, est le menu *Refactor*. Il peut à partir d'un littéral quelque part dans son code, introduire un variable statique contenant cette valeur.



## Avant de commencer

À ce stade, vous avez un projet fonctionnel sans structure MVC. Si vous utilisez `git` et voulez conserver cet état, vous pouvez faire un *commit* de votre travail et *taguer* (mettre un *tag*, une étiquette) cette version.

Avec ce *tag*, il sera facile de revenir à cette version si nécessaire.

En quelques commandes :

```
git add .
git commit -m "Add non mvc solution"
git tag -a "v1" -m "Version without mvc"
```

## 3 Le modèle, la vue et le contrôleur

### 3.1 La classe `Model`, point d'entrée du modèle

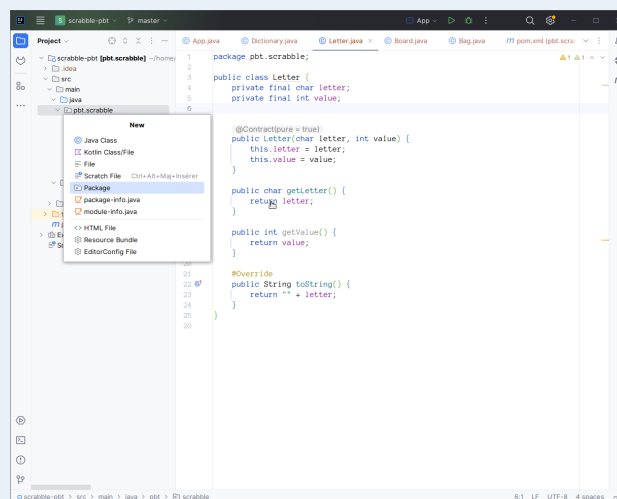
Parmi les classes métiers — celles « qui font le boulot » de manipuler les données, fournir des résultats, de connaître les règles — nous trouvons les classes `Letter`, `Direction`, `Bag` et `Dictionary`. Ces classes contiennent plutôt les données de l'application.

Créez un `package` `g12345.scrabble.model` dans votre projet et glissez / déposez ces 4 classes dans ce nouveau package en acceptant le *refactoring* proposé dans une fenêtre modale.

Des `import` vont apparaître dans certaines classes. C'est normal.

#### Tip

Pour ajouter, une classe ou un package (voir ci-dessus), le raccourci `[Ctrl][N]` sous linux et macOS (`[Ctrl][Alt][Ins]` sous MS Windows) est très utile. Il ouvre le menu contextuel *New in this directory*.



Si vous avez déjà ajouté la vérification des mots, c'est-à-dire ajouté la règle : un mot doit être dans le dictionnaire du Scrabble pour pouvoir être ajouté sur le plateau du jeu alors vous l'avez probablement fait dans la méthode `setLetters` de la classe `Board`.

Ce n'est pas vraiment de la responsabilité du plateau de jeu de faire cette vérification : le plateau du jeu est responsable des lettres qu'il contient tandis que le dictionnaire est responsable de vérifier la validité d'un mot. La vérification de la validité d'un

mot avant l'ajout de celui-ci sur le plateau est une autre responsabilité que nous allons déléguer à une classe qui sera responsable du jeu. Nous allons ajouter une classe `g12345.scrabble.model.Model` qui jouera ce rôle.

C'est cette classe qui aura la responsabilités du jeu.

Cette classe sera aussi l'interlocutrice privilégiée du contrôleur.

Déplacez la classe `Board` dans le *package* `g12345.scrabble.model` nous y reviendrons plus tard et créez la classe `Model`.

Une fois la classe créée :

- ✍ ajoutez-lui 5 attributs : un *board*, un *bag*, un *dictionary*, un score et un chevalet (*rack*) contenant les lettres du joueur ou de la joueuse ;
- ✍ écrivez un constructeur sans paramètre qui permet de créer une nouvelle partie : un plateau de jeu, un sac de lettres, un dictionnaire, un score nul et un chevalet **déjà rempli** à l'aide du sac préalablement créé ;
- ✍ la classe n'a qu'un seul *getters* — `getScore()` — et pas de *setters*.

### Rappel

L'encapsulation nous demande de garder nos attributs privés et de ne les exposer que si nécessaire.

Le modèle ne donne pas ses attributs `board` et `rack` « *comme ça* » pour qu'ils ne soient pas modifiables en dehors du modèle.

— D'accord. C'est pour respecter l'encapsulation. Mais comment la vue par exemple pourra accéder aux lettres du *board* ?

— Nous allons lui donner une méthode `Lettre get(int row, int col)` qui retourne une lettre à la fois.

— Ah j'ai compris. C'est mieux parce que la vue ne pourra pas modifier le *board* mais aura quand même accès à chaque lettre.

— C'est bien ça.

- ✍ Ajoutez une méthode `Lettre get(int row, int col)` à la classe `Model` qui retourne la lettre à la position donnée.

— Et pour le *rack* (la liste de lettres), on fait la même chose ?

— Non, pour le *rack*, nous allons retourner une liste **non modifiable**.

— On donne la liste mais on ne peut pas la modifier. Malin.

— Oui, comme ceci :

```
public List<Lettre> getRack() {  
    return Collections.unmodifiableList(rack);  
}
```

— C'est donc bien un accesseur pour le *rack* ?

— Oui et la classe `Model` a choisit comment elle voulait l'exposer.

Le modèle doit pouvoir proposer de jouer et jouer ; c'est placer des lettres sur le plateau.

- ✍ Dans la classe `Model`, ajoutez une méthode

```
void setLetters(int row, int col, Direction d, int[] indexes).
```

Oui, elle porte le même nom que celle de la classe `Board` mais ne fera pas la même chose et n'a pas les mêmes paramètres. Le dernier paramètre représente les indices des lettres du chevalet — dans l'ordre — que le joueur voudrait placer.

Cette méthode tentera de placer les lettres sur le plateau de jeu :

- ▷ elle vérifie que les lettres formeront un mot valide en interrogeant la classe `Dictionary` ;
- ▷ elle essaie ensuite de placer les lettres sur le plateau de jeu en demandant à la classe `Board` de le faire (avec sa méthode homonyme) ;
- ▷ si tout se passe bien, elle place les lettres et remplit le chevalet du joueur sinon, elle lance une exception.

### Remarque

La méthode aurait pu avoir comme dernier attribut un tableau de lettres comme dans la classe `Board` plutôt qu'un tableau d'indices... dans ce cas le modèle aurait du vérifier que les lettres proposées se trouvent bien dans le chevalet du joueur.

Comme l'on passe les indices des lettres sur le chevalet, pas besoin de faire cette vérification.

Le modèle est maintenant fonctionnel. Passons à la vue.

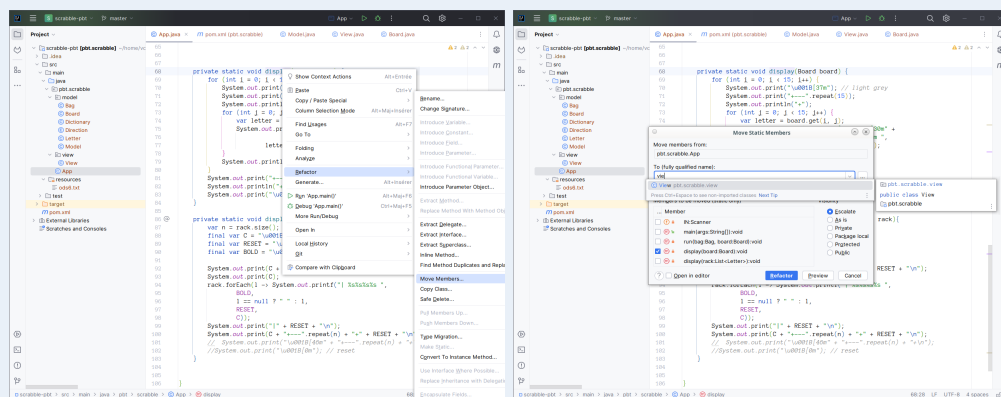
## 3.2 La classe View, une première vue du modèle

Cette classe aura comme attribut le modèle qu'elle reçoit comme paramètre de son constructeur afin de pouvoir obtenir du modèle les données qu'elle doit afficher.

Déplacez les deux méthodes `display` — `display(Board)` et `display(List<Letter>)` — se trouvant actuellement dans la classe principale `App`, rendez les non statiques et renommez-les en `displayBoard()` et `displayRack()`.

### Tip

La combinaison de touche **[Ctrl][M]** est un raccourci pour ouvrir le menu contextuel *Refactor / Move static Members*. Ce menu propose de déplacer un membre — par exemple une méthode — dans une autre classe dès lors que le curseur se trouve sur le nom du membre à déplacer.



La vue ne fera rien d'autres que faire les affichages lorsque le contrôleur lui demande de le faire.



**Remarque** Vous pouvez choisir si le score est affiché en même temps que le plateau de jeu ou si vous ajoutez une méthode `displayScore()`.

### 3.3 La classe **Controller**, le contrôleur

Le contrôleur prendra la responsabilité de la boucle du jeu et des entrées de l'utilisateur ou de l'utilisatrice.

#### MVC non GUI

Nous mettons ici en œuvre le design MVC sans interface graphique (*GUI, graphical user interface*). Nous n'avons pas de clic de souris ou autres interactions « graphiques » avec l'utilisateur ou l'utilisatrice. Nous déléguons la responsabilité des entrées utilisateur ou utilisatrice au contrôleur.

À ce stade, votre classe **App** ne conserve qu'une méthode *main* à l'allure suivante :

```
public static void main( String[] args ) {
    System.out.println( "SCRABBLE" );
    Bag bag = new Bag();
    Board board = new Board();
    run(bag, board);
}
```

La responsabilité de la classe **App** est simplement de lancer l'application avec MVC. Son code aura maintenant l'allure suivante :

```
public static void main( String[] args ) {
    System.out.println( "SCRABBLE" );
    Model model = new Model();
    View view = new View(model);
    Controller controller = new Controller(model, view);
    controller.run();
}
```

- ▷ L'instantiation d'un modèle.
- ▷ L'instantiation d'une vue recevant le modèle dont elle a besoin.
- ▷ L'instantiation d'un contrôleur recevant le modèle et la vue puisqu'il fait le lien entre les deux.
- ▷ Une méthode **run** dans le contrôleur qui contiendra la boucle du jeu.

Pour que ce code fonctionne :

- ✍ ajoutez deux attributs à la classe **Controller** : un modèle et une vue ;
- ✍ ajoutez un constructeur à deux paramètres ;
- ✍ rendez la méthode **run** non statique et supprimez ses paramètres ;
- ✍ adaptez la méthode **run** pour que l'ensemble soit fonctionnel.

Vous trouverez à la figure 4 [page suivante](#) un diagramme de classes représentant le projet en fin de refactorisation.

La refactorisation de votre code avec une architecture MVC est maintenant terminée. Jouez !

---

‡. Les liens entre **Controller** et **Model** / **View** ainsi que ceux entre **Model** et **Board** / **Bag** / **Dictionary** représentent une **agrégation**.

Dès lors que, par exemple, la classe **Model** est agrégée au **Board**, un attribut de type **Board** est présent dans la classe **Model**. Il ne devrait donc pas être réécrit dans le diagramme de la classe **Model**. Nous l'avons représenté ici dans un souci de lisibilité.

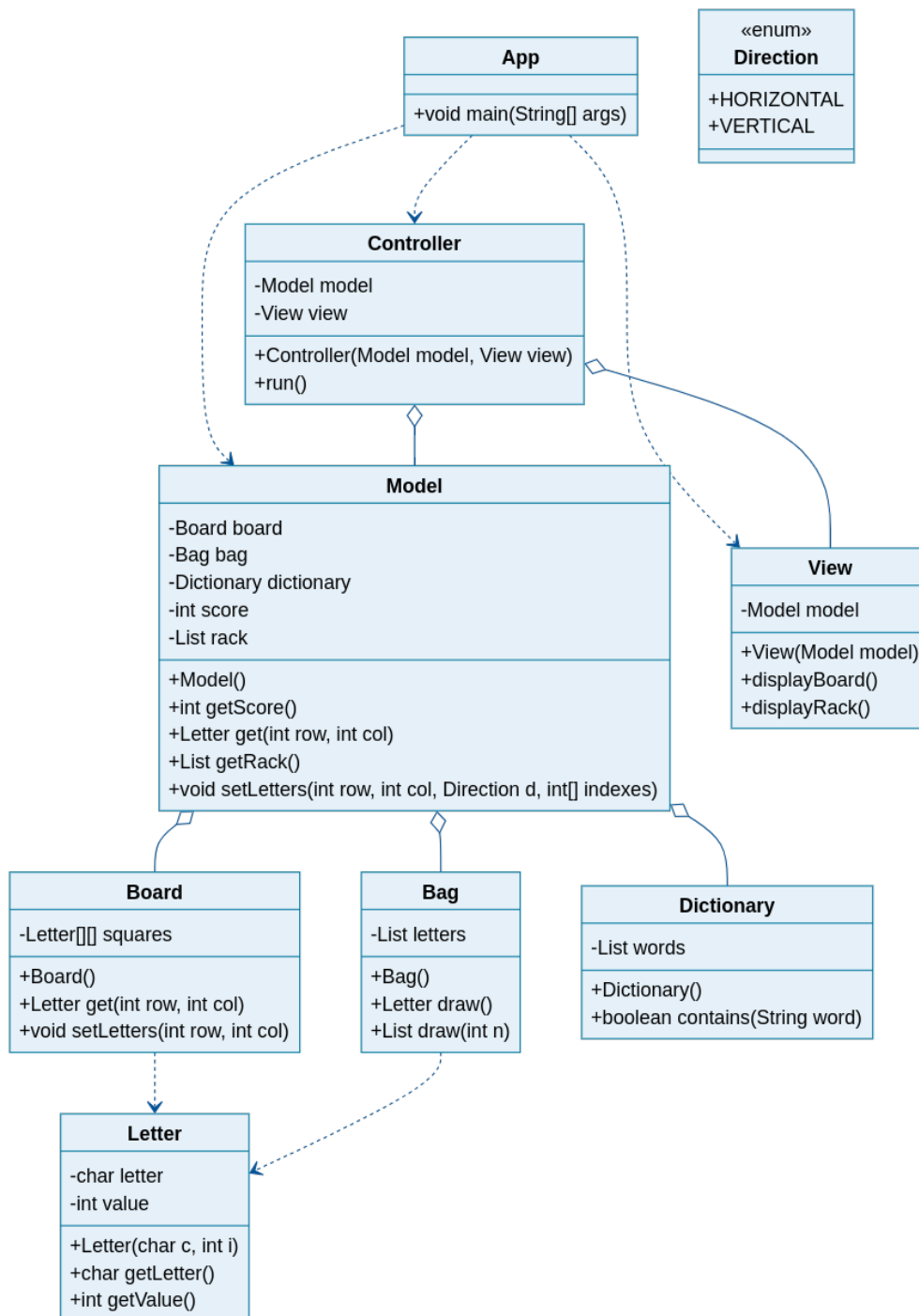


FIGURE 4 – Diagramme de classes<sup>‡</sup>

## 4 Une autre vue

Nous voudrions avoir une autre vue que la vue actuelle en minimisant les changements de code. Pour ce faire :

- ✍ demandez le code de la classe **View** de votre voisin ou voisine et ajoutez-le à votre projet sans modifier son nom complet § ;
- ✍ modifiez **uniquement** votre classe principale en remplaçant la ligne  
`import votrelogin.scrabble.view.View;` par  
`import leloginduvoisin.scrabble.view.View;`

... le code devrait immédiatement être fonctionnel.

---

§. Pour transmettre du code à son voisin, vous pouvez le faire par mail, *via* votre *gdrive* ou en utilisant des outils en ligne (un peu *legacy*) de *pastebin*. Par exemple [paste.esigoto.info](https://paste.esigoto.info)